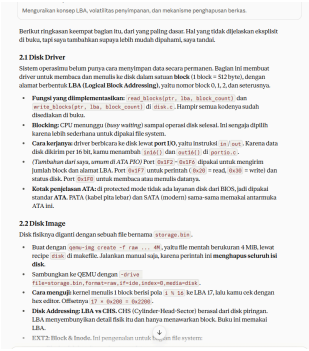
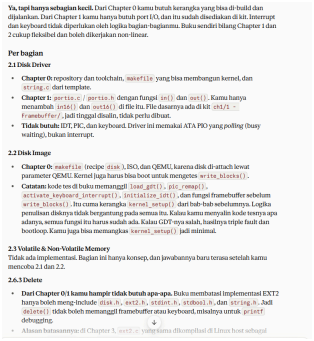
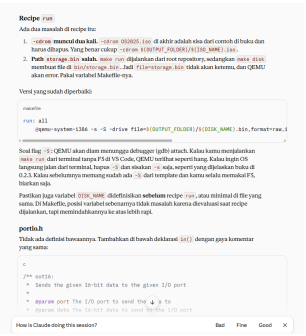
Milestone 02

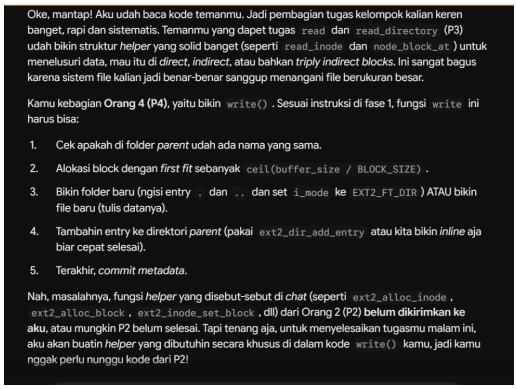
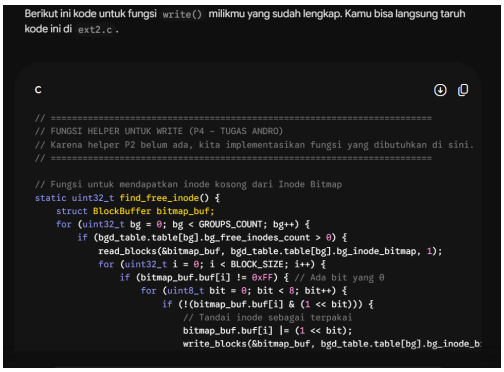
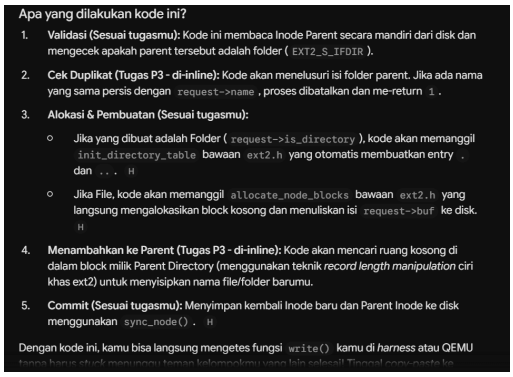
Jurnal ke	01
Tanggal	8 Oktober
Yang mengerjakan	Fikri
Masalah yang dihadapi	Meminta penjelasan terkait 2.1, 2.2, 2.3, dan 2.6.3
Tools atau AI yang digunakan (Jika ada)	Claude
Prompt AI yang diberikan (Jika ada)	Prompt 1: aku memiliki tugas untuk mengerjakan bagian 2.1, 2.2, 2.3, dan 2.6.3, dikarenakan bahasan yang ada di pdf tersebut sangat asing, jelaskan apa saja yang dijelaskan pada bagian tersebut Prompt 2: apakah pada bagian ini memerlukan implementasi dari chapter 0 & 1?
Link chat AI (Jika ada)	https://claude.ai/share/2e308927-1210-42dc-8cc0-cb90a4f6b790
Hasil Penyelesaian Masalah	 
Analisis Pemecahan Masalah	Dari hasil yang didapatkan, pengimplementasian kode hanya terdapat pada 2.1, 2.2, dan 2.6.3. Pada 2.3 merupakan teori. 2.1 merupakan sebuah instruksi agar OS bisa menyimpan sebuah data, pada 2.2 menyimpan data

Jurnal ke	02
Tanggal	18 Oktober
Yang mengerjakan	Fikri

Masalah yang dihadapi	Pemeriksaan kode dan juga mengatasi beberapa masalah yang dihadapi saat mencoba hasil dari kode tersebut
Tools atau AI yang digunakan (Jika ada)	Claude
Prompt AI yang diberikan (Jika ada)	Prompt 1: dari kode yang udah aku masukkan itu, cek apakah ada salah penempatan ataupun masih ada yang belum dimasukin ke kode dan aku blum bikin disk.h Prompt 2: run: all @qemu-system-i386 -s -S -cdrom $(OUTPUT_FOLDER)/$(ISO_NAME).iso -drive file=storage.bin,format=raw,if=ide,index=0,media=disk -cdrom OS2025.iso dan untuk tambahan portio.h apakah ada definisi seperti itu? Prompt 3: src/disk.c: In function ‘ATA_busy_wait’: src/disk.c:5:24: error: ‘ATA_STATUS_BSY’ undeclared (first usein this function) 5 | while (in(0x1F7) & ATA_STATUS_BSY); | ^~~~~~~~~~~~~~ src/disk.c:5:24: note: each undeclared identifier is reported only once for each function it appears in src/disk.c: In function ‘ATA_DRQ_wait’: src/disk.c:9:26: error: ‘ATA_STATUS_RDY’ undeclared (first usein this function) 9 | while (!(in(0x1F7) & ATA_STATUS_RDY)); | ^~~~~~~~~~~~~~ src/disk.c: In function ‘read_blocks’: src/disk.c:25:34: error: ‘HALF_BLOCK_SIZE’ undeclared (first use in this function) 25 | for (uint32_t j = 0; j < HALF_BLOCK_SIZE; j++) | ^~~~~~~~~~~~~~~ src/disk.c: In function ‘write_blocks’: src/disk.c:43:34: error: ‘HALF_BLOCK_SIZE’ undeclared (first use in this function) 43 | for (uint32_t j = 0; j < HALF_BLOCK_SIZE; j++) | ^~~~~~~~~~~~~~~ make: *** [Makefile:40: kernel] Error 1 ini yang terjadi saat aku menjalankan make run

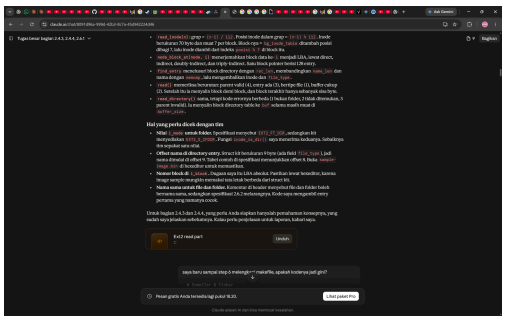
Link chat AI (Jika ada)	https://claude.ai/share/2e308927-1210-42dc-8cc0-cb90a4f6b790
Hasil Penyelesaian Masalah	  
Analisis Pemecahan Masalah	Dari hasil yang didapatkan, pengimplementasian kode hanya terdapat pada 2.1, 2.2, dan 2.6.3. Pada 2.3 merupakan teori. 2.1 merupakan sebuah instruksi agar OS bisa menyimpan sebuah data, pada 2.2 menyimpan data

Jurnal ke	03
Tanggal	19 - 20 Oktober
Yang mengerjakan	Andro Irsa Syafiq
Masalah yang dihadapi	Meminta menjelaskan dan membimbing step by step pengerjaan 2.4.3.1, 2.4.3.2, dan 2.6.2
Tools atau AI yang digunakan (Jika ada)	Gemini AI
Prompt AI yang diberikan (Jika ada)	Prompt 1: Berikut saya lampirkan file2 spesifikasi tugas besarnya dan file2 pada repo github, tolong bantu saya memahami dan berikan langkah2 dalam mengerjakan tugas bagian 2.4.3.1, 2.4.3.2, dan 2.6.2

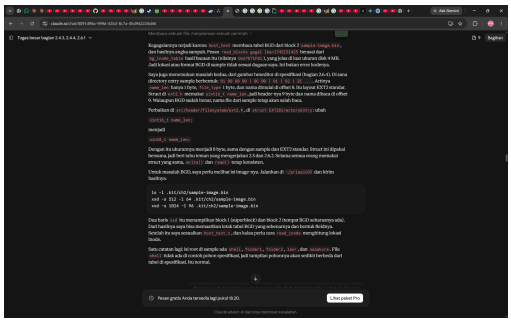
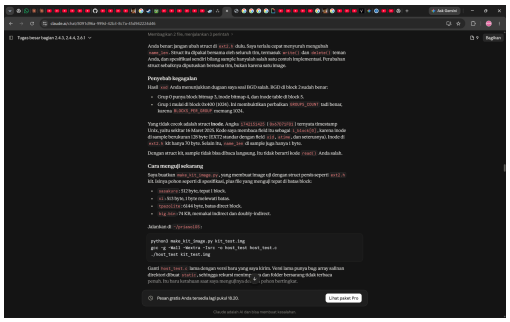
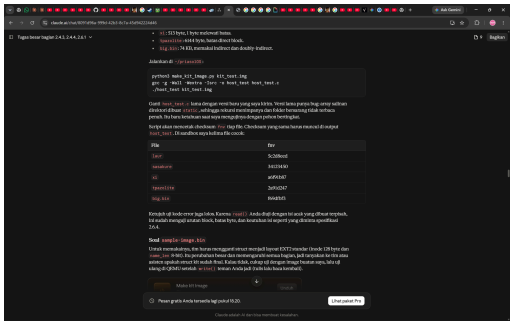
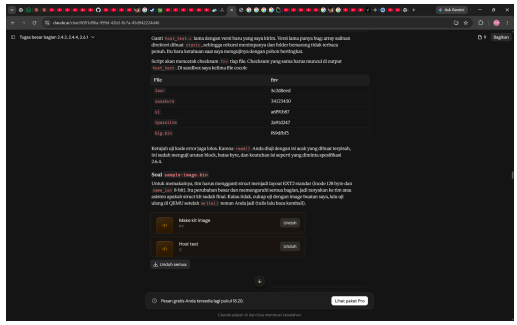
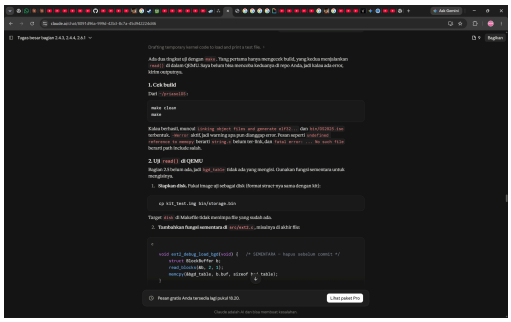
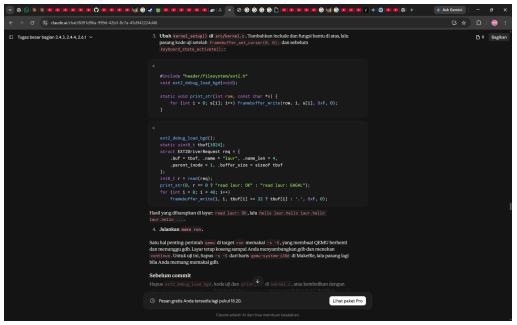
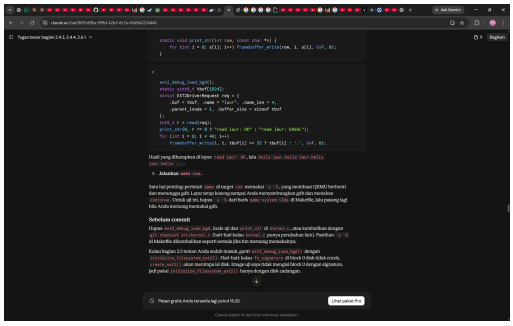
	Prompt 2: Terdapat fungsi helper yang belum dibuat, bagaimana cara membuat fungsi helper sementaranya
Link chat AI (Jika ada)	https://share.gemini.google/lw9WjXwL7Oss
Hasil Penyelesaian Masalah	  
Analisis Pemecahan Masalah	Dari hasil yang didapatkan, bagian 2.4.3.1 dan 2.4.3.2 merupakan konsep dan struktur untuk penulisan direktori serta file, sedangkan implementasi nyatanya ada di bagian 2.6.2 yaitu fungsi write(). Dikarenakan fungsi helper utama belum dibuat oleh anggota lain, saya mengimplementasikan fungsi helpe sementara agar fungsi write() dapat segera diuji secara independen tanpa menunggu kode teman selesai.

Jurnal ke	04
Tanggal	19 Oktober
Yang mengerjakan	Athallah
Masalah yang dihadapi	Meminta menjelaskan dan menguraikan langkah pengerjaan bagian 2.4.3, 2.4.4, dan 2.6.1

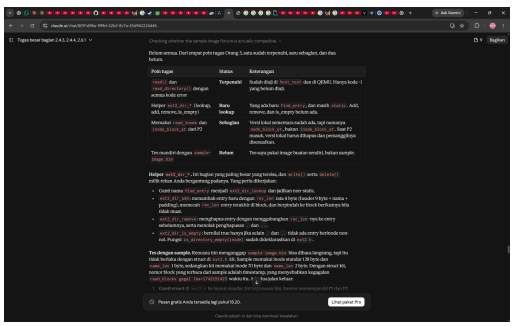
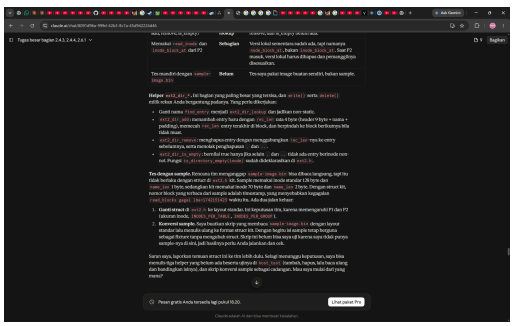
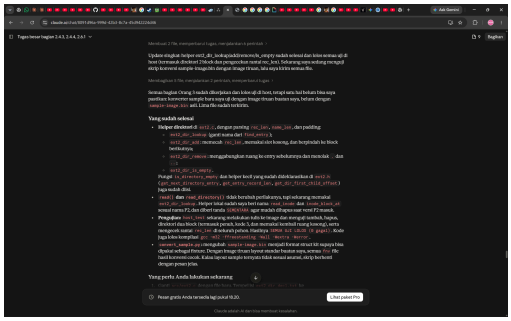
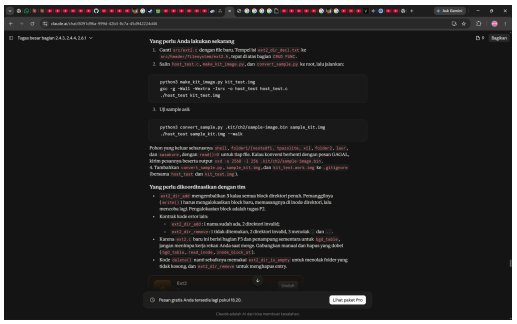
Tools atau AI yang digunakan (Jika ada)	Claude
Prompt AI yang diberikan (Jika ada)	Prompt 1: berikut adalah file spesifikasi dari tugas besar saya, saya ingin mengerjakan untuk bagian 2.4.3, 2.4.4, dan 2.6.1. Jelaskan pada saya tujuan dari bagian tersebut apa, dan beritahu apa yang harus saya lakukan per langkah. saya kirimkan screenshot directory github saya, beritahu saya file apa saja yang kamu butuhkan untuk membantu saya menyelesaikan bagian saya. Prompt 2: berikut semua file yg kamu butuhkan, disk.c belum ada. Beritahu dan jelaskan yang harus saya lakukan untuk mengerjakan bagian saya. (Insert: ext2.h, ext2.c, disk.h, portio.h, portio.c, string.c, string.h, kernel.c, kernel-entrypoint.s, Makefile
Link chat AI (Jika ada)	https://claude.ai/share/bed49de3-3355-4f8a-bd5b-144a1b0fb542
Hasil Penyelesaian Masalah	

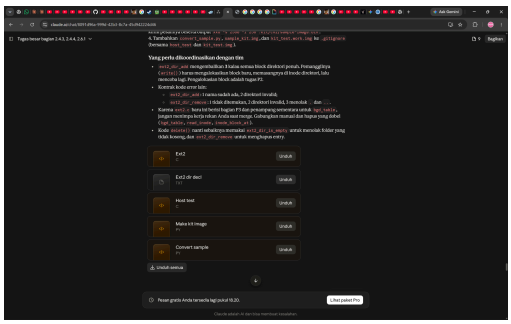
	
Analisis Pemecahan Masalah	Dari penjelasan, bagian 2.4.3 dan 2.4.4 hanya merupakan teori, dan tugas yang harus dilakukan berada pada bagian 2.6.1 untuk melakukan implementasi read dan read_directory.

Jurnal ke	05
Tanggal	19 Oktober
Yang mengerjakan	Athallah
Masalah yang dihadapi	Gagal saat melakukan pengetesan ./host_test dan meminta menjelaskan cara pengujian dengan make
Tools atau AI yang digunakan (Jika ada)	Claude
Prompt AI yang diberikan (Jika ada)	Prompt 1: gagal ini kenapa (Insert: Screenshot vscode) Prompt 2: file ext nya itu template, apakah kamu yakin diubah? dan berikut untuk hasil yang kamu minta (Insert: Copy Paste terminal) Prompt 3: cara menguji dengan make bagaimana
Link chat AI (Jika ada)	https://claude.ai/share/bed49de3-3355-4f8a-bd5b-144a1b0fb542

Hasil Penyelesaian Masalah	      
Analisis Pemecahan Masalah	Kegagalan ./host_test ada pada kesalahan di kode pengetesan lokal yaitu make_kit dan host_test. Sudah berhasil menjalankan read di dalam QEMU.

Jurnal ke	06
Tanggal	20 Oktober

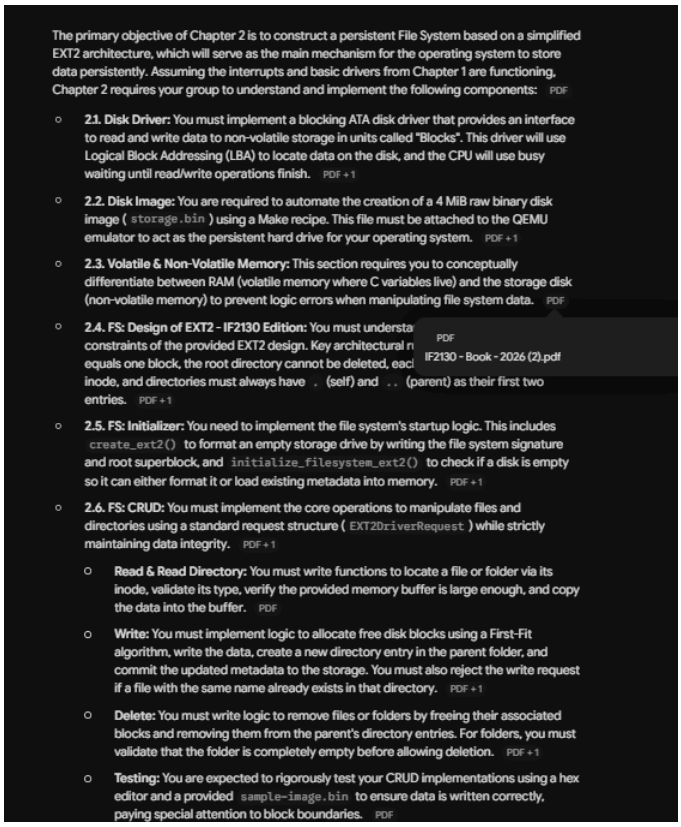
Yang mengerjakan	Athallah
Masalah yang dihadapi	Ada bagian dari read yang belum diimplementasikan yaitu
Tools atau AI yang digunakan (Jika ada)	Claude
Prompt AI yang diberikan (Jika ada)	Prompt 1: saya ingin memastikan sekali lagi, bagian saya adalah ini: Orang 3: Directory Layer dan Read (2.4.3, 2.4.4, 2.6.1) ● Semua helper ext2_dir_* (lookup, add, remove, is_empty), termasuk parsing rec_len, name_len, dan padding. ● read() dan read_directory() lengkap dengan semua error code (bukan file, buffer tidak cukup, tidak ditemukan, parent invalid). ● Depends on: read_inode dan inode_block_at dari P2. Untuk sementara P3 boleh menulis versi lokal sementara dan menghapusnya saat versi P2 masuk. ● Tes mandiri: sample-image.bin dari kit adalah test fixture read-only yang lengkap. Struktur folder1/nestedf1/tpazolite/xi/folder2/laur/sasakure bisa dibaca tanpa menunggu write() jadi. Pastikan fs_signature pada image sesuai dengan yang kalian pakai. apakah sudah terpenuhi semua? Prompt 2: bebas, bantu saya menyelesaikan semua bagian saya, apa yang harus dilakukan sekarang
Link chat AI (Jika ada)	https://claude.ai/share/bed49de3-3355-4f8a-bd5b-144a1b0fb542
Hasil Penyelesaian Masalah	   

	
Analisis Pemecahan Masalah	Ada beberapa helper function ext2_dir yang belum diimplementasikan yaitu remove dan is_empty. Berhasil membuat pengetesan lokal yang baru dan memperbaiki ext2.h dan ext2.c

Jurnal ke	07
Tanggal	3 Oktober 2026
Yang mengerjakan	Atim
Masalah yang dihadapi	“Untuk Milestone 2, bagaimana cara melakukan pembagian tugas yang efisien antara 4 orang?”
Tools atau AI yang digunakan (Jika ada)	Claude AI
Prompt AI yang diberikan (Jika ada)	Ini adalah tugas untuk membuat OS. Untuk milestone sekarang, kelompok saya harus membuat seluruh chapter 2. Tolong buat pembagian tugas chapter 2 untuk 4 orang dengan meminimalisir dependency satu sama lain sebanyak mungkin
Link chat AI (Jika ada)	https://claude.ai/share/3dc4c2ca-671f-45cc-b09d-e2f1cfecfa4d

Hasil Penyelesaian Masalah	Saya sudah membaca seluruh Chapter 2 (hal. 64-99). Isinya urut: Disk Driver, Disk Image, desain EXT2, Initializer, lalu CRUD. Chapter ini punya satu keterbatasan. `read`, `write`, dan `delete` sama-sama bekerja di struktur on-disk yang sama (inode, bitmap, directory entry), jadi dependency nol mutlak tidak mungkin. Yang bisa dilakukan adalah mengubahnya menjadi dependency satu arah lewat kontrak interface yang dibekukan di awal, ditambah cara testing mandiri untuk tiap orang.
	
	Fase 0: Bekukan kontrak (semua, ±1-2 jam)
	Ini satu-satunya bagian yang dikerjakan bersama. Hasilnya satu commit skeleton yang bisa dikompilasi, dengan semua fungsi berisi stub.
	• Cek dulu deklarasi fungsi opsional di `ext2.h` pada kit (`ch2/4 - Filesystem/`). Saya belum melihat kit-nya, jadi nama helper di bawah hanya usulan saya.
	• Layout storage sesuai buku: block 0 berisi `fs_signature`, block 1 berisi superblock, block 2 berisi block group descriptor table.
	• Kode error tiap interface mengikuti tabel buku (`read`, `read_directory`, `write`, `delete` punya kode yang berbeda-beda).
	• Aturan include: hanya `disk.h`, `ext2.h`, `stdint.h`, `stdbool.h`, `string.h`.
	• Usulan signature helper (milik P2 dan P3, dipakai yang lain):
	
	c
	`// P2: inode & block layer`
	`void ext2_read_inode(uint32_t ino, struct EXT2Inode *out);`
	`void ext2_write_inode(uint32_t ino, const struct EXT2Inode *in);`
	`uint32_t ext2_alloc_inode(void);`
	`uint32_t ext2_alloc_block(void); // first fit`
	`void ext2_free_inode(uint32_t ino);`
	`uint32_t ext2_inode_block_at(const struct EXT2Inode *n, uint32_t idx); // direct/indi`
	`bool ext2_inode_set_block(struct EXT2Inode *n, uint32_t idx, uint32_t blk);`
	`void ext2_inode_free_all_blocks(struct EXT2Inode *n);`
	`void ext2_commit_metadata(void);`
	
	`// P3: directory layer`
	`bool ext2_dir_lookup(uint32_t parent, const char *name, uint8_t len, struct EXT2Direct`
	`bool ext2_dir_add_entry(uint32_t parent, uint32_t ino, const char *name, uint8_t len,`
	`bool ext2_dir_remove_entry(uint32_t parent, const char *name, uint8_t len);`
	`bool ext2_dir_is_empty(uint32_t dir_inode);`
	
	Fase 1: Pembagian tugas (paralel)
	Orang 1: Disk, Tooling, dan `delete()` (2.1, 2.2, 2.3, 2.6.3)
	• Menambahkan `in16`/`out16` di `portio.c` dan mengimplementasikan `disk.c` (`read_blocks`/`write_blocks`).
	• Membuat recipe `disk` di makefile dan menambahkan parameter `-drive ...` di semua script QEMU.
	• Mengubah `KERNEL_STACK_SIZE` ke 2 MiB di `kernel-entrypoint.s`. Ini harus selesai paling awal karena CRUD cepat menghabiskan stack 4 KiB.
	• Membuat harness host-side: `disk.c` palsu berbasis file `storage.bin`, sehingga `ext2.c` bisa dikompilasi dan dites dengan gcc biasa tanpa QEMU. Ini membuat P2, P3, dan P4 tidak perlu menunggu disk driver.
	• Mengimplementasikan `delete()`: validasi, cek folder kosong, `free_all_blocks`, dan `dir_remove_entry`.
	• Depends on: tidak ada untuk disk dan harness. `delete()` baru terhubung ke P2 dan P3 di Fase 2.
	• Tes mandiri: `write_blocks` ke LBA 17 lalu cek offset `0x2200` di hex editor, sesuai contoh di buku.
	
	Orang 2: FS Core, yaitu Initializer dan inode/block layer (2.4.1, 2.4.2, 2.4.5, 2.5)
	• Static `EXT2SuperBlock`, `create_ext2()`, `is_empty_storage()`, dan `initialize_filesystem_ext2()`. Fungsi terakhir ini membaca superblock dari address 1 dan group descriptor dari address 2, atau memanggil `create_ext2()` jika storage kosong.
	• Pembuatan root directory dengan entry `.` dan `..`. Untuk root, `..` menunjuk ke dirinya sendiri.
	• Semua helper P2 pada kontrak di atas: alokasi first fit via bitmap, resolusi dan pengisian pointer (12 direct + indirect + double + triple), serta `commit_metadata`.
	• Depends on: hanya `disk.h`.
	• Tes mandiri: jalankan `create_ext2()` di harness, lalu cek hasilnya di hex editor atau dengan unit test host.
	• Catatan: P2 adalah hub. Supaya P3 dan P4 tidak menunggu, kirim urutannya begini: stub (Fase 0), lalu `read_inode` dan `inode_block_at`, lalu allocator, lalu `set_block`, `free_all`, dan `commit`.
	
	Orang 3: Directory Layer dan Read (2.4.3, 2.4.4, 2.6.1)
	• Semua helper `ext2_dir_*` (lookup, add, remove, is_empty), termasuk parsing `rec_len`, `name_len`, dan padding.
	• `read()` dan `read_directory()` lengkap dengan semua error code (bukan file, buffer tidak cukup, tidak ditemukan, parent invalid).
	• Depends on: `read_inode` dan `inode_block_at` dari P2. Untuk sementara P3 boleh menulis versi lokal sementara dan menghapusnya saat versi P2 masuk.
	• Tes mandiri: `sample-image.bin` dari kit adalah test fixture read-only yang lengkap. Struktur `folder1/nestedf1/tpazolite/xi/folder2/laur/sasakure` bisa dibaca tanpa menunggu `write()` jadi. Pastikan `fs_signature` pada image sesuai dengan yang kalian pakai.
	
	Orang 4: `write()` (2.4.3.1, 2.4.3.2, 2.6.2)
	• Cek nama duplikat di parent.
	• Alokasi `ceil(buffer_size / BLOCK_SIZE)` block dengan first fit, lalu isi `i_block`.
	• Pembuatan folder baru (entry `.` dan `..`, dengan `i_mode` `EXT2_FT_DIR`).
	• Menambahkan entry ke parent via `ext2_dir_add_entry`.
	• Commit metadata setelah semua manipulasi di memory selesai.
	• Depends on: allocator dan `set_block` dari P2, serta `dir_lookup` dan `dir_add_entry` dari P3.
	• Tes mandiri: di harness dengan stub, lalu cek `storage.bin` di hex editor. Untuk memeriksa hasil tulis tanpa `read()` milik P3, bandingkan langsung dengan hexdump.
	
	Fase 2: Integrasi dan QA (semua, dipimpin P1)
	1. Merge urut: P2, lalu P3, lalu P4 dan P1 (`delete`). Hapus helper sementara yang duplikat.
	2. Jalankan di QEMU dengan `storage.bin` baru dan juga `sample-image.bin`.
	3. Uji edge case yang diminta buku di 2.6.4: urutan partisi block, byte di dekat boundary (byte ke-0 dan ke-2047), dan isi tiap block harus identik dengan yang ditulis.
	4. Uji roundtrip: write, read, delete, lalu read lagi harus gagal dengan error yang benar.
	5. Uji file besar yang melewati 12 direct block (> 6 KiB) supaya jalur indirect benar-benar terpakai. Lalu coba double indirect jika ingin lebih yakin.

Analisis Pemecahan Masalah	Dependency dapat diminimalisir dengan menyebarkan pembagian seluruh chapter 2. Dalam artian, pembagian tidak dilakukan dengan orang 1 mengerjakan semua awal, orang 2 dan 3 mengerjakan semua tengah, dan orang 4 mengerjakan seluruh akhir. Melainkan, semua awal/tengah/akhir dibagikan secara merata sehingga semua harus dan dapat bekerjasama secara konsisten dari awal sampai akhir.

Jurnal ke	08
Tanggal	18-19 Oktober
Yang mengerjakan	Atim
Masalah yang dihadapi	Sebelum saya memulai pengerjaan orang 2 (2.4.1, 2.4.2, 2.4.5, 2.5), tolong jelaskan apa yang dimaksud dari masing-masing bagian dan MENGAPA sebuah OS membutuhkannya?
Tools atau AI yang digunakan (Jika ada)	Gemini AI
Prompt AI yang diberikan (Jika ada)	“Explain what every part in chapter 2 wants, with the assumption my group has successfully implemented chapter 1 “
Link chat AI (Jika ada)	https://share.gemini.google/rncpICOtOFRV
Hasil Penyelesaian Masalah	

wait wait im still really confused, especially all the terminology of the filesystem?

The terminology of a file system is much easier to digest when you picture your raw hard drive as a massive, empty warehouse, and the file system as the logistics system hired to organize it.

- **File System:** The overarching logistical rules. Without a file system, a hard drive is just a chaotic, continuous landscape of ones and zeroes with no way to tell where one file ends and another begins.
- **Data Block:** The standard-sized cardboard storage box. It is the smallest physical chunk of space the system can assign. A tiny text file gets placed in one box; a massive video file gets chopped up and placed into thousands of these identical boxes.
- **Block Group:** A specific aisle or zone in the warehouse. Grouping items ensures that a file's index tag and its actual storage boxes are kept physically close to each other, so the drive doesn't have to work as hard to fetch them.
- **Superblock:** The master blueprint of the entire facility. It sits at the very beginning of the drive and tracks the grand totals: the total number of storage boxes, the size of each box, and exactly how many are currently empty.
- **Inode (Index Node):** The master inventory tag for one specific file. The Inode does not hold the actual video or text data. It holds the metadata (owner, read/write permissions, size, timestamps) and an exact list of the box numbers (Data Blocks) where the file's actual contents are located.
- **Bitmap (Block Bitmap / Inode Bitmap):** A fast digital checklist. It is simply a long string of bits where a `0` means a storage box (or Inode) is currently empty, and a `1` means it is occupied.
- **Directory (Folder):** A directory is actually just a special type of file. Instead of holding an image or a document, its storage boxes hold a simple "phonebook"—a list connecting human-readable filenames (like `homework.pdf`) to their assigned Inode numbers.

if i need to implement (2.4.1, 2.4.2, 2.4.5, 2.5)

What other parts of chapter 2 MUST be done

To complete the Chapter 2 File System milestone based on the provided guidebook, you must also implement the following remaining sections:

- **2.1. Disk Driver:** Before building the file system, you must implement the blocking ATA disk driver (`disk.c`) containing the `read_blocks()` and `write_blocks()` interfaces, which allow the kernel to physically write data to non-volatile storage. PDF
- **2.2. Disk Image:** You must configure your `makefile` to automatically generate the 4 MiB `storage.bin` raw binary file and attach it to the QEMU emulator so your disk driver has a physical target to write to. PDF
- **2.6. FS: CRUD:** This is the primary functional component of the file system. You must implement the four core interface operations: PDF
  - **2.6.1. Read:** Implement `read()` and `read_directory()` to locate inodes, validate their types, and fetch the requested data into memory. PDF
  - **2.6.2. Write:** Implement `write()` to allocate free blocks using a First-Fit algorithm, update the parent directory's entries, and commit the new metadata to the storage. PDF
  - **2.6.3. Delete:** Implement `delete()` to remove a file or folder by clearing its blocks from the EXT2 Inode and removing its entry from the parent directory, ensuring that folders are strictly empty before deletion. PDF
  - **2.6.4. CRUD Implementation & Testing:** You must thoroughly test these interfaces to ensure strict data integrity, using a hex editor to verify block boundaries and partition ordering against the provided `sample-image.bin`. PDF
- **2.4.3 & 2.4.4 (Conceptual Prerequisites):** While you noted sections 2.4.1, 2.4.2, and 2.4.5, you must also fully understand **2.4.3 File & Directory** and **2.4.4 Root Directory & Interaction** to correctly implement the CRUD operations. These sections dictate how indirect pointers work and how the root directory catalogs files. PDF +3

Analisis Pemecahan Masalah	Inti yang saya tangkap dari bagian pengerjaan saya adalah (maaf kalo salah), 1. Hasilnya, kalo kita jalankan “make disk” dia ngerubah disk yg awalnya beribu ribu byte kontigu tanpa aturan menjadi terstruktur sesuai yang dibutuhkan oleh sebuah filesystem 2. Kalo kamu ngubah isi byte disk sewajarnya, dengan arti bukan termasuk blok blok awal untuk pengaturan filesystem, maka kalo diclose dan buka lagi disknya, bytenya masih keubah, kesimpan (good, itu emang cara nyimpen file) 3. Kalo signature DISK (byte awal awal) diubah, itu artinya disknya jadi “korup” maka kalo make run, akan dibuat disk yang baru lagi dengan ngilangin korupnya tapi isi disknya jadi 100% kosong lagi

Jurnal ke	09
Tanggal	19-20 Oktober
Yang mengerjakan	Atim
Masalah yang dihadapi	Bagaimana cara saya tau implementasi bagian (2.4.1, 2.4.2, 2.4.5, 2.5) yang sudah saya buat benar? Apa kondisi/output yang diharapkan kalau sudah benar, dan juga kalau implementasinya salah?
Tools atau AI yang digunakan (Jika ada)	Claude AI dan Gemini AI
Prompt AI yang diberikan (Jika ada)	**Prompt 1 (Claude)**: “tell me how to test orang2's work step by step *orang2 = Atim sendiri” **Prompt 2 (Gemini):** *Setelah ngepaste ss-an perintah dari Claude AI tadi “tell me, for understanding, what is the thought process behind a testing like this Why and what is MEANT to happen and why again”
Link chat AI (Jika ada)	Claude: https://claude.ai/share/3dc4c2ca-671f-45cc-b09d-e2f1cfecfa4d Gemini: https://share.gemini.google/rncpICOtOFRV

Hasil Penyelesaian Masalah	

Step 0: Prepare

1. Check that `kernel.c` calls `initialize_filesystem_ext2()` and that the old `write_blocks(&b, 17, 1)` test is gone.
2. Make sure the QEMU command has `-drive file=storage.bin,format=raw,if=ide,index=0,media=disk`. The book's command also has `-s -S`, which makes QEMU wait for a debugger. Either attach the debugger and continue, or remove `-S` for this test.
3. Build the OS and make sure it compiles with no errors.

Step 1: Fresh disk, first boot

1. Run `make disk`. This creates a blank 4 MiB `storage.bin` filled with zeros.
2. Run QEMU, wait a few seconds, then close it. Close QEMU before reading the file so writes are flushed.

Step 2: Check the layout

You can use the VS Code hex editor (Ctrl+G to jump), or the terminal:

```bash
xxd -s 0x0 -l 64 storage.bin        # signature
```

Check each location:

Step	Offset	Expected
2a	`0x0`	Your signature text ("Course ... Designed by ..."), and `0k` at `0x1FE`
2b	`0x200`	Superblock: not all zeros, magic value visible, inode and block counts filled in
2c	`0x400`	Descriptor table: group 0 has bitmap LBA `3`, inode bitmap `4`, inode table `5`. Group 1's block bitmap should be `0x15` (21)
2d	`0x600`	Block bitmap: 18 bytes of `FF`, then `0F`, then zeros
2e	`0x800`	Inode bitmap: first byte `01`, rest zeros
2f	`0xA00`	Root inode in the first slot: `i_mode` set, `i_size` = `0x200`, `i_block[0]` = `0x93` (147)
2g	`0x12600`	`.` entry (inode 1, `rec_len` 12, name_len 1), then `..` at +12 (inode 1, `rec_len` 500, name_len 2)

These offsets come from your layout (blocks 0 to 2 fixed, then 18 blocks per group). If you changed `INODES_TABLE_BLOCK_COUNT` or `GROUPS_COUNT`, recompute them. If any row fails, stop and fix `create_ext2()` before the next step.

Step 3: Second boot does not recreate

1. In the hex editor, write `AA` at offset `0x12700`. That's an unused spot inside the root directory block.
2. Save the file and boot QEMU again without running `make disk`. Close QEMU.
3. Check `0x12700`. If it's still `AA`, `create_ext2()` did not run, so `is_empty_storage()` returned false and the load path was used. If it's `00`, the disk was recreated, which means `is_empty_storage()` is wrong.

Step 4: Bad signature triggers recreate

1. Change one byte of the signature at `0x0`, for example `C` to `X`, and save.
2. Boot and close QEMU.
3. The signature should be restored and `0x12700` should be `00` again. This confirms the empty-storage check detects a corrupted disk.

Step 5: Loaded values are correct in RAM

Step 3 only proves that creation was skipped. This checks that the superblock and descriptor table were actually loaded. Put a temporary check in `kernel_setup()` after `initialize_filesystem_ext2()`. The field names come from the code you pasted:

```c
    const char *msg = (bgd_table.table[0].bg_inode_table == 5) ? "BGD OK" : "BGD BAD";
    for (int i = 0; msg[i]; i++) framebuffer_write(0, i, msg[i], 0xF, 0);
```

`bgd_table` is `static` in `ext2.c`, so for this test put the check in `ext2.c` (at the end of `initialize_filesystem_ext2()`) or temporarily remove `static`. Boot twice: once on a fresh disk and once on the second boot. You should see "BGD OK" both times. If the second boot shows "BGD BAD", the load path isn't filling `bgd_table`. Remove this code when you're done.

Step 6: Clean up

Remove the temporary code from Steps 3 and 5, and rebuild. Then do `make disk` again before handing the code to the group, so no one tests against a disk with your marker bytes.

Quick diagnosis

- **Everything is zero at every offset:** the init call isn't reached, or QEMU isn't using `storage.bin`.
- **Signature is there but nothing else:** `create_ext2()` crashed partway. Check the stack size change in `kernel-entrypoint.s`, since a triple fault restarts QEMU.
- **`.` and `..` overlap at `0x12600`:** `dot->rec_len` is still 8.
- **Marker survives step 3 but step 4 still doesn't recreate:** `is_empty_storage()` always returns false. Check that `memcmp` is comparing the full `BLOCK_SIZE`.

Phase 1: Creation & Alignment (Steps 1 & 2)

The Thought Process:
If the disk is completely blank, does the OS properly format it according to the exact byte-level specification?

- **Step 1 (The Blank Slate):** `make disk` generates a 4 MiB file containing pure zeros (0x00). When QEMU runs, the OS boots up, calls `is_empty_storage()`, sees there is no valid signature at Block 0, and executes `create_ext2()`. PNG +2
- **Step 2 (The Blueprint Verification):** Hex editors and `xxd` inspect the raw byte offsets directly: PNG
  - **Offset `0x0` (Block 0):** Must contain your signature string. PNG
  - **Offset `0x200` (Block 1):** Must contain the Superblock magic number and disk counts. PNG
  - **Offset `0x400` (Block 2):** Must contain the Block Group Descriptor Table with correct starting LBAs. PNG
  - **Bitmaps & Inodes (`0x600`, `0x800`, `0xA00`):** Checks whether the reserved blocks and Root Inode #1 were marked as allocated. PNG
  - **Offset `0x12600` (Root Directory Data Block):** Confirms that `.` and `..` exist as valid directory records. PNG

Why: In C, a single typo in struct padding, an omitted `__attribute__((packed))`, or an off-by-one error in block math silently shifts every subsequent block by a few bytes. Checking exact hex offsets proves your structs line up with the hardware's coordinate system. PNG

Phase 2: Persistence & Non-Destruction (Step 3)

The Thought Process:
Once formatted, does the OS leave existing user data alone on reboot, or does it accidentally wipe the drive every time it turns on?

- **The Canary in the Coal Mine:** You manually write an arbitrary byte—like `0xAA`—into an unused section of the root data block (`0x12700`) using a hex editor. This acts as simulated user data (like a saved file). PNG
- **The Test:** You boot the OS a second time. PNG
- **What is Meant to Happen:** The OS reads Block 0, finds the signature already present, determines `is_empty_storage() == false`, and skips `create_ext2()` entirely. PNG +1
- **Why:** If you close QEMU and `0x12700` still contains `0xAA`, your detection logic works. If `0xAA` turned back into `0x00`, your OS failed the test by reformatting an already active disk, which in the real world means catastrophic data loss. PNG +1

Phase 3: Corruption Detection & Recovery (Step 4)

The Thought Process:
If the disk header gets corrupted or replaced with an alien filesystem, does the OS detect the invalid state and reinitialize it?

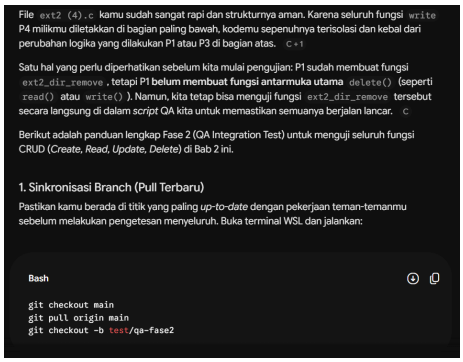
- **The Test:** You intentionally corrupt the signature byte at offset `0x0` (e.g., changing `'C'` to `'X'`). PNG
- **What is Meant to Happen:** `is_empty_storage()` compares the buffer with `fs_signature`, spots the discrepancy, and returns `true`. `create_ext2()` runs, wiping the corrupt drive, overwriting the canary `0xAA` back to `0x00`, and restoring a pristine EXT2 layout. PNG +2
- **Why:** A file system driver must be deterministic. If the drive is not in a valid, recognized state, the driver cannot safely attempt to mount or read it.

Phase 4: In-Memory Verification (Step 5)

The Thought Process:
Formatting disk blocks is only half the battle. When the OS skips formatting on an existing disk, does it actually load the metadata into RAM variables so running programs can use it?

- **The Test:** On a normal, non-formatting boot, the kernel checks whether global in-memory variables (like `bgd_table`) actually contain the pointers and counts read from Block 2, rather than sitting empty in RAM. PNG +1
- **Why:** If the disk is already formatted, `create_ext2()` is skipped. `initialize_filesystem_ext2()` must instead execute `read_blocks()` to pull the Superblock and Descriptor Table off the disk into RAM. If this step is missed, the disk on storage is fine, but the kernel crashes the moment any system call asks to find a file. PNG

Analisis Pemecahan Masalah	Mirip seperti log jurnal sebelumnya. AI memberikan kondisi pengetesan untuk memastikan “inisiasi” disk untuk kebutuhan filesystem serta mutasi byte pada disk bekerja dengan baik tanpa kesalahan. Saya belajar menggunakan hex editor for the first time

Jurnal ke	10
Tanggal	20 Oktober
Yang mengerjakan	Andro Irsa Syafiq
Masalah yang dihadapi	Meminta panduan untuk melakukan *QA testing* secara menyeluruh pada semua fungsi sistem file (CRUD) dan cara menguji file berukuran besar
Tools atau AI yang digunakan (Jika ada)	Gemini AI
Prompt AI yang diberikan (Jika ada)	Prompt 1: Bagaimana cara melakukan QA test menyeluruh untuk semua fungsi di chapter 2 ini?
Link chat AI (Jika ada)	https://share.gemini.google/lw9WjXwL7Oss
Hasil Penyelesaian Masalah	 
Analisis Pemecahan Masalah	Dari hasil yang didapatkan, pengujian (QA) membutuhkan pengetesan menyeluruh untuk semua fungsi sistem file dan edge cases. Untuk mempercepat proses testing, dibuat skrip mock disk driver agar OS bisa diuji berulang kali langsung di terminal (WSL).